

Modifiability

LG Architecture Training Program

Paulo Merson

Modifiability

- How easy it is to make changes to the system
- The ability to easily create and change code requires
 - Readability (of the code)
 - Modularity (high cohesion and loose coupling)
 - Testability
- It applies to evolutionary and corrective maintenance
- It's a **development-time** quality; it's not a runtime quality of the system



Modifiability tactics

- Split module
- Restrict dependencies
- Increase semantic coherence (aka SRP)
- Redistribute responsibilities (aka Refactor)

Writing clean code is what you must do in order to call yourself a professional. [Martin08]



Robert C. Martin
(Uncle Bob)



Split module

- Simply break a module into smaller modules
- The goal is to reduce the size of the modules and **increase cohesion**
- A larger module
 - is harder to read and understand
 - is more likely to be modified by 2+ people at the same time
 - is more prone to conflicts and untimely deployment
 - takes longer to load when the system is executed

This tactic applies to a single source file but also to modules that contain other modules

Piecemeal growth can lead to a big ball of mud [Foote97]



Brian Foote

Restrict dependencies

- A dependency (static or dynamic) between software elements represent **coupling**
- Coupling refers to the degree of interdependence between software elements
 - It's also the probability that a modification to one module will propagate to the other [Bass21]
- The goal is not to remove coupling altogether; coupling is essential to build software
- The goal is to limit coupling to the minimum necessary

*We want **loose coupling**
(aka low coupling)*

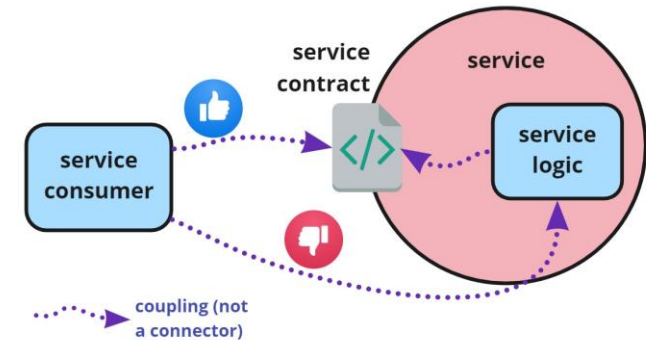


*We avoid **tight coupling**
(aka high coupling)*

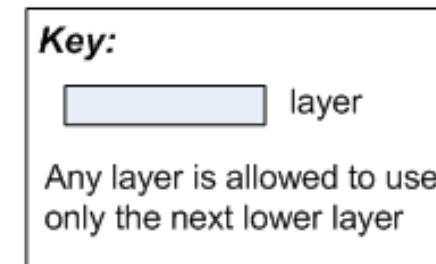
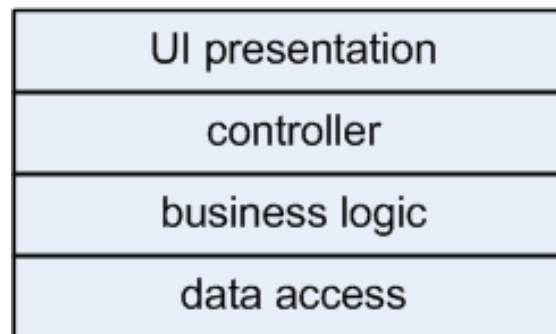


How to restrict dependencies

- A module should only make visible what is needed by clients
 - Corollary: clients should not “bypass” the interface



- Different design strategies can be used to restrict dependencies, such as
 - published contracts/interfaces
 - wrapper components
 - layered architectures



Increase semantic coherence

- Different responsibilities should be in different modules
 - Corollary: the same module should not have more than one functionality (ideally)
- Complements the Redistribute Responsibilities tactic (described later)

Remember that a given functionality may be handled in multiple modules (or layers)

The Single Responsibility Principle (SRP) is about cohesion, which is the functional relatedness of the elements of a module



Redistribute responsibilities

*If the same (or very similar) functionality exists in two modules, then this functionality should be **factored** into a single location*

- Important: the same functionality may be diluted across different modules per the separation of concerns principle
 - This separation is legitimate and there's no need to refactor

This tactic was called Refactor in [BASS13]

Refactoring is (...) a way to restructure software to make design insights more explicit, (...) to clarify the software architecture, and to prepare to make additions easier. [Fowler99]



Martin Fowler

Redistribute responsibilities (avoid duplication) + Increase semantic coherence (SRP) example

- Scenario:
 - For our smart TVs there is a library module that lists available apps ordered by app name
 - I need a module that lists available apps ordered by category
 - I search the code base and find the existing module
 - The option to order by category is not available on that module
- What to do?

Implement the logic inside some module of *my system*



Create a *new module* with the new functionality



Negotiate with the custodian of the existing module to add the new functionality





? question @10 Ⓞ ★ 🔒

10 views

Actions ▾

Inquiry on tactics and patterns for enhancing accuracy in quality attributes

Hello

I am seeking information on effective tactics and patterns for improving the accuracy quality attribute in system design and implementation. Accuracy is crucial for ensuring that the system outputs are precise and reliable.

Any detailed information, case studies, or references on these tactics and patterns would be highly appreciated.

Thank you for your assistance.

Best regards,
Hyunsoo Kim

Accuracy (in AI and ML)

- Refers to the proportion of predictions or classifications a model makes that are correct
- It's a metric to gauge how well a model performs
- For binary classifications, $\text{accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$

Accuracy is a limited metric because imbalanced datasets can lead to biased models



Accuracy tactics

- Increase sampling rate (ex.: read sensor more frequently, increase FPS)
- Augment data sample
 - Enhance the diversity of data used to train the model
- Ensemble learning
 - Combine predictions of multiple models (bagging is a technique for that)
- Convolutional Neural Networks (CNN)
- Residual networks (ResNets)
- K-fold cross-validation
- Custom loss function
- Generative Adversarial Network (GAN)



Paulo Merson <paulomerson@gmail.com> Jun 5, 2024, 6:08 PM (14 hours ago) ☆ 😊 ↩ ⋮
to Rick, Len ▾

Hi Rick, hi Len,

I hope all is well with you. 😊

I'm teaching an intensive course on Software Architecture at CMU and using SAP4 as a textbook. We're covering different QAs (performance, availability, security, usability, modifiability) and selected tactics. One of the students asked about tactics for Accuracy. (In their project they need to implement image recognition and he used accuracy in that context.)

A couple of questions to you:

- 1) Do you see Accuracy as a quality attribute? I know in practice what's important is to have a clear QA scenario and not the classification, but in class (and in a book) it's still a relevant question, especially in this age of data analysis. Looking at SAP4, I see accuracy related to reliability and "functional suitability".
- 2) Can you name any design tactics or patterns for Accuracy? 🤔

Thanks in advance for your thoughts on this!

Paulo

Len Bass Wed, Jun 5, 6:36 PM (13 hours ago) ☆ 😊 ↩ ⋮
to me, Rick ▾

I didn't answer your question about tactics. Choose a different model is one tactic. Improve the training data set is another.

Not architectural, I admit.

Len

On Wed, Jun 5, 2024 at 6:33 PM Len Bass <len.bass@gmail.com> wrote:

Accuracy is an AI quality attribute. It measures how well a model performs at prediction.

I don't know about scenarios but there are some metrics used to measure it. This is a paragraph from our book

Model accuracy testing: The model accuracy is tested using the selected test cases, which comprise a test suite. A test suite can include both test datasets that mirror real-world complexities and evaluation metrics or benchmarks for quantifying accuracy. Metrics might be general, like precision and F1 score, or specific to a domain, such as BLEU for language translation.

Rick Kazman

12:22 AM (8 hours ago)



to Len, me ▾

I agree. This is not architectural. It is an implementation detail of the chosen ML component. I could choose different text classification algorithms for example, with different F1 scores, and typically nothing else in the system would be affected.

Regards

rick

Rick Kazman

Danny and Elsa Lui Distinguished Professor of Information Technology Management

University of Hawaii, Honolulu, HI 96825

Len Bass

7:15 AM (1 hour ago)



to Rick, me ▾

Methods for improving the performance of a system with respect to AI qualities seem to be process oriented rather than architecturally oriented. This is true of accuracy, security, human values. Less so for availability.

Len

AI/ML terms (1)



- A* Search
- Analysis of Variance (ANOVA)
- Annealing
- Approximate K-NN
- ARIMA model
- Automatic Speech Recognition (ASR)
- Bagging
- Baidu PaddlePaddle
- Bayesian statistics
- BERT (Bidirectional Encoder Representations from Transformers)
- BLEU
- Catastrophic forgetting
- Categorical crossentropy
- ChrF metric
- CIFAR-10
- Class imbalance
- Cluster analysis
- ColBERT
- Convolutional Neural Networks (CNN)
- Cosine-Similarity
- Custom loss function
- Deep learning
- Deep Reinforcement Learning
- Direct Preference Optimization (DPO)
- Edge TPU
- Embeddings
- Ensemble methods
- F1 Score (Harmonic Mean of Precision and Recall)
- Few-shot prompting

AI/ML terms (2)



- Fuzzy c-means (FCM)
- Generative Adversarial Network (GAN)
- Gradient-boosted decision tree (GBDT)
- Gradient boosting machine (GBM)
- Graph neural network (GNN)
- Group Relative Policy Optimization (GRPO)
- Hierarchical Clustering
- Hierarchical RNN
- Hierarchy-aware Knowledge Graph (HAKG)
- Hungarian Algorithm
- Jupyter Notebook
- K-fold cross-validation
- K-means
- K-medoids
- Keras
- Knowledge Graph-based Intent Network (KGIN)
- KPI anomaly detection
- Kullback–Leibler (KL) divergence
- Learning rate warm-up
- Llama
- Long short-term memory (LSTM)
- LoRA (Low-Rank Adaptation)
- Lyapunov stability
- Mamba
- Mean Average Precision (mAP)
- Mean squared error
- METEOR metrics
- Mistral AI
- MMLU
- Monte Carlo simulation

AI/ML terms (3)



- Muon
- Naïve Bayes Classifier
- Named entity recognition (NER)
- Neural Architecture Search (NAS)
- Neural Combinatorial Optimization
- Neural network
- NLLB
- NLTK
- NN model
- Normalized Discounted Cumulative Gain (NDCG)
- Numpy
- Overfitting
- Pandas
- Parquet file
- Phi-3
- Poisson Cluster
- Proximal Policy Optimization (PPO)
- Pytorch
- Quantization
- Receiver Operating Characteristic - Area Under Curve (ROC-AUC)
- Reinforcement Learning (RL)
- Reinforcement Learning from AI Feedback (RLAIF)
- Reinforcement Learning from Human Feedback (RLHF)
- Residual networks (ResNets)
- Retrieval-augmented generation (RAG)
- RNN
- Rotary Position Embedding (RoPE)
- ROUGE scores
- RWKV

AI/ML terms (4)



- Sentiment analysis
- Supervised Fine Tuning SFT
- Shallow neural network
- SHAP analysis
- Small language models (SLMs)
- SMOTE
- Spectral Clustering
- Stochastic Gradient Descent (SGD)
- Support vector machine (SVM)
- Temperature
- TensorFlow
- TOPSIS
- Top_p (nucleus sampling)
- Top@5 accuracy
- Ultrafeedback dataset
- Viterbi algorithm
- Weak learners
- Weibull distribution
- Whisper
- Word2Vec
- YOLO framework
- Zero-shot chain-of-thought (CoT)



? question @9

14 views

Actions

About Performance Tactic: Manage work requests

Dear, Paulo!

Thank you for today's lecture.

On today's lecture, we've learned about the tactics of Performance including "Manage work requests".

But I didn't understand why this tactic can be used to achieve Performance.

For example, API quota and Request throttling limit requests, which will reduce the throughput of the system but ensure the availability of system.

(we usually set the quota to a lower value rather than the maximum capacity of the system.)

Therefore I think this tactic is more considerable about Availability than Performance.

How do I utilize this tactic for Performance?

Thank you in advance.

lectures

Edit good question | 0

Updated 2 days ago by Bokyoung Ku

Manage work requests

- Reduce the number of requests coming into the system
- **Manage sampling rate**
 - Adjust sampling frequency to the response times the computing element offers
 - Ex.: sensor data sampling rate; FPS in video capture or video processing
 - Goal is to balance predictable latency and good enough quality/fidelity
- **Manage event arrival**
 - **API quota**: limit the requests per client per day, week or month to a quota
 - **Rate limiting**: limit the number of requests per client per seconds or per minute
 - **Request throttling**: confine requests to configured limits to avoid overwhelming the service

The token bucket algorithm is often used for request throttling





to Rick, Len ▾

Rick, Len,

I hope all is well. 😊

I'm teaching the LG architecture program and, as you know, SAP 4 is a textbook.

This week we covered Performance tactics. One of them was *Manage work requests*, which is subdivided into *Manage event arrival* and *Managing sampling rate*. That is on page 139 of the book.

- As common examples of *Manage event arrival*, in class I talk about rate limiting, API quota and request throttling, which are common features in API gateways and other middleware platforms these days. These features are responsible for issuing the infamous HTTP 429 "Too Many Requests" error to prevent application services from getting overwhelmed.
- As an example of Managing sampling rate I mention Netflix and Youtube, that can adjust the FPS/resolution based on bandwidth and available CPU/memory.

After class, one of the students, Bokyoung Ku, cleverly pointed out that this tactic many times aims at protecting the system rather than improving performance. Here's what he wrote on Piazza:

On today's lecture, we've learned about the tactics of Performance including "Manage work requests".

But I didn't understand why this tactic can be used to achieve Performance.

For example, API quota and Request throttling limit requests, which will reduce the throughput of the system but ensure the availability of system.

(we usually set the quota to a lower value rather than the maximum capacity of the system.)

Therefore I think this tactic is more considerable about Availability than Performance.

How do I utilize this tactic for Performance?

In class we discuss that QA taxonomies have limited practical use. The important thing is to properly specify QA rqmts (using QA scenarios for example) rather than classifying them.

In any case, I'm inclined to agree with Bokyoung that *Manage work requests* contributes much more to availability than to performance/throughput. I'd love to know your thoughts.

Paulo

Paulo Merson

Rick Kazman

to Paulo, Len ▼

Wed, Jun 4, 5:04 PM (13 hours ago)



Hi Paulo,

Allow me to try to convince you (and Bokyoung). :-)

In reasoning about performance you can think about how to increase resources seamlessly, so that work done scales with work submitted. This is the promise of much of the cloud architecture. This is the "Manage Resources" sub-category of tactics.

But you do not always have that option. In an embedded system, for example, you have whatever resources you have, and **your goal as an architect is to do the most work with what you have**. That is where the "Control Resource Demand" sub-category comes in. Managing work requests is one of the options that an architect has. If I don't have enough resources to handle every event, it makes sense to handle the high priority ones (or handle them first).

Think about the network of roads in a city. Adding new roads and new lanes is very costly and seldom done. So most performance tactics used in urban design are design to control resource demand. Consider:

- HOV lanes,
- highway entrances with stop lights where a car is "released" into the highway every five seconds,
- inner-city areas where only certain cars are allowed,
- the fact that you need to pull over to the right when an ambulance is behind you

Each of these "highway tactics" is about **using the resources that you have more efficiently.**

So I disagree: This has little to do with Availability and everything to do with Performance.

Cheers,

rick

Len Bass

to Rick, Paulo ▼

Wed, Jun 4, 9:35 PM (9 hours ago)



To add to what Rick said. Think about requests arriving from multiple sources. From the perspective of one client, throttling may slow you down but from the perspective of a different client, **throttling may allow you requests to be serviced in situations where they may not have been previously.**

As Rick said, with limited resources, it is an aspect of performance to keep as many clients performing as possible.

Len



That's all I



Questions?

Paulo Merson
pmerson@cmu.edu